

# Local versus Global Allocation Policies

## 1 Core Idea

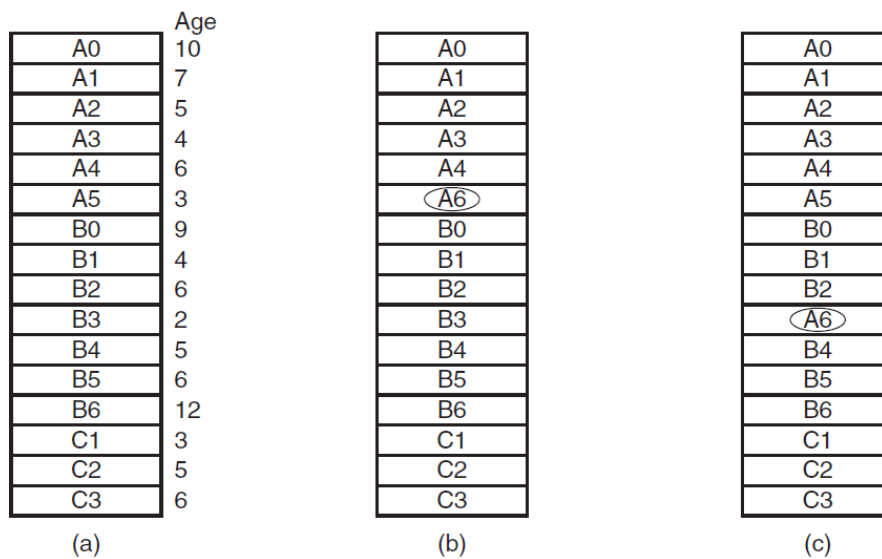
### Problem

When a page fault occurs, the operating system must decide whether the victim page must come from the faulting process or may come from any process.

This is a memory-allocation policy question. It is separate from the replacement algorithm itself.

| Policy                    | Meaning                                                            |
|---------------------------|--------------------------------------------------------------------|
| <b>Local replacement</b>  | Replace only a page owned by the faulting process.                 |
| <b>Global replacement</b> | Replace any page in memory, even if it belongs to another process. |

## 2 Local versus Global Example



**Figure 3-22.** Local versus global page replacement. (a) Original configuration. (b) Local page replacement. (c) Global page replacement.

### Figure Setup

Processes *A*, *B*, and *C* are runnable. Process *A* gets a page fault and needs to load page *A6*.

In the original configuration, each page has an age value. Lower age means the page was used more recently.

| Panel | Meaning                                                             |
|-------|---------------------------------------------------------------------|
| (a)   | Original memory configuration with pages from $A$ , $B$ , and $C$ . |
| (b)   | Local replacement: $A6$ replaces one of $A$ 's own pages.           |
| (c)   | Global replacement: $A6$ replaces the best victim from all memory.  |

### 3 Local Replacement

With local replacement, process  $A$  can evict only one of its own pages.

From the figure,  $A$ 's pages have ages:

| Page | $A0$ | $A1$ | $A2$ | $A3$ | $A4$ | $A5$ |
|------|------|------|------|------|------|------|
| Age  | 10   | 7    | 5    | 4    | 6    | 3    |

If the algorithm chooses the least recently used page among only  $A$ 's pages, it chooses  $A5$  because  $A5$  has the smallest age value shown for  $A$ 's local set.

#### Local Result

In panel (b),  $A6$  replaces  $A5$ . Other processes keep all their frames.

### 4 Global Replacement

With global replacement, the victim can belong to any process.

From all pages in the figure, the lowest age is:

$$B3 = 2$$

So the global algorithm chooses  $B3$ , even though process  $A$  caused the page fault.

#### Global Result

In panel (c),  $A6$  replaces  $B3$ . Process  $A$  gains a frame, and process  $B$  loses one.

### 5 Main Difference

| Policy | Strength                                                 | Weakness                                              |
|--------|----------------------------------------------------------|-------------------------------------------------------|
| Local  | Protects processes from losing frames to others.         | Fixed allocation can cause thrashing or waste memory. |
| Global | Dynamically shifts memory toward processes that need it. | One process can take frames away from another.        |

#### General Rule

Global algorithms often work better because process working-set sizes change over time.

## 6 Why Local Allocation Can Fail

Local replacement effectively gives each process a fixed memory share.

|                            |                                                                   |
|----------------------------|-------------------------------------------------------------------|
| <b>Working-set change</b>  | Local-policy problem                                              |
| <b>Working set grows</b>   | The process may thrash even if other processes have spare frames. |
| <b>Working set shrinks</b> | The process may keep frames it no longer needs, wasting memory.   |

## 7 Allocating Frames Among Processes

If global allocation is used, the OS must decide how many frames each process should get.

### 7.1 Equal Allocation

Divide frames equally among runnable processes.

#### Example

If 12,416 available frames are shared among 10 processes, each process receives:

$$\left\lfloor \frac{12,416}{10} \right\rfloor = 1241$$

frames, with 6 frames left for a free pool.

Equal allocation is simple, but it ignores process size.

### 7.2 Proportional Allocation

Allocate frames in proportion to process size.

#### Why Proportional Allocation Helps

A 300-KB process usually needs more frames than a 10-KB process. Equal allocation gives them the same number, which is often unreasonable.

Even with proportional allocation, each process should receive some minimum number of frames so it can execute basic instructions.

## 8 Minimum Frame Requirement

Some machines require multiple pages for a single instruction.

#### Minimum Frames

A single instruction may need pages for the instruction itself, the source operand, and the destination operand. If these cross page boundaries, several pages may be needed at once.

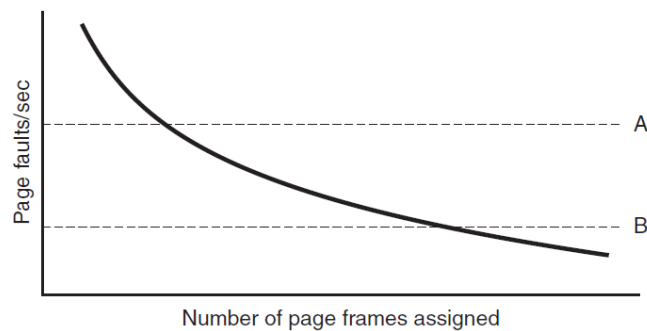
Therefore, allocating too few frames can make a process unable to execute at all.

## 9 Page Fault Frequency

### PFF

**Page Fault Frequency (PFF)** controls how many frames a process should have by monitoring its page fault rate.

PFF does not choose the victim page directly. It controls the size of the process's allocation set.



**Figure 3-23.** Page fault rate as a function of the number of page frames assigned.

## 10 Reading the PFF Figure

The graph shows that page fault rate usually decreases as a process receives more page frames.

| Line            | Meaning                                                                              |
|-----------------|--------------------------------------------------------------------------------------|
| A               | Fault rate is too high. Give the process more frames.                                |
| B               | Fault rate is very low. The process may have too many frames. Take some frames away. |
| Between A and B | Fault rate is acceptable. Keep allocation roughly unchanged.                         |

### Simple Running Mean

$$\text{new mean} = \frac{\text{old mean} + \text{faults in last second}}{2}$$

## 11 Local/Global Is Independent of Some Algorithms

Some replacement algorithms can be used either locally or globally.

| Algorithm | Local version                                       | Global version                          |
|-----------|-----------------------------------------------------|-----------------------------------------|
| FIFO      | Oldest page owned by faulting process.              | Oldest page in all memory.              |
| LRU       | Least recently used page owned by faulting process. | Least recently used page in all memory. |

## 12 Algorithms That Are Naturally Local

Working-set-based algorithms refer to a specific process.

### Working Set Is Per Process

There is no useful “working set of the whole machine.” The working set concept is tied to one process’s locality of reference.

Therefore, working set and WSClock are naturally local strategies. Taking the union of all processes’ working sets would lose the locality information that makes the method useful.

## 13 Final Summary

### One-Box Summary

Local replacement evicts only from the faulting process.  
Global replacement can evict from any process.  
Global allocation adapts better to changing working sets.  
PFF adjusts frame allocation using page fault rate.  
Working-set methods are naturally process-local.